中山大学计算机学院

人工智能

本科生实验报告

课程名称：Artificial Intelligence

学号		姓名	

一、 实验题目

启发式搜索

二、 实验内容

1. 算法原理

1.2 A*算法原理

A*算法是一种启发式搜索算法，使用评估函数 f(n) = g(n) + h(n)来指导搜索方向：
- **g(n)**：从初始状态到当前状态的实际代价（移动步数）
- **h(n)**：从当前状态到目标状态的估计代价（启发式函数）
- **f(n)**：总估计代价

A*算法使用优先队列，每次扩展 f 值最小的节点，保证找到最优解（当 h(n)可采纳时）。

1.3 IDA*算法原理

IDA*（Iterative Deepening A*）是 A*的改进版本，结合了深度优先搜索的节省内存和 A*的启发式指导：

1. 设置初始阈值为 h(初始状态)
2. 进行深度优先搜索，当 f(n) > 阈值时回溯
3. 记录所有超过阈值的 f(n)值，取最小值作为新阈值
4. 重复步骤 2-3 直到找到解

优点：
- 内存占用小（只需 O(d)空间，d 为搜索深度）
- 适合解决状态空间巨大的问题
- 保证找到最优解

1.4 启发式函数设计

（1）曼哈顿距离（Manhattan Distance）

$$h(n) = \sum_{i=1}^{15} (|x_i - x_{goal}| + |y_i - y_{goal}|)$$

- **可采纳性**：是，永远不会高估实际代价
- **一致性**：是，满足三角不等式
- **计算复杂度**：O(1)（常数 15 次计算）

（2）错位棋子数（Misplaced Tiles）
$$h(n) = \sum_{i=1}^{15} [tile_i \neq goal_i]$$

- **可采纳性**：是
- **一致性**：是
- **精确度**：较低，不考虑移动距离

（3）线性冲突（Linear Conflict）
$$h(n) = h_{manhattan}(n) + 2 \times conflicts$$

线性冲突：两个数字在同一行（列），且它们的目标位置也在同一行（列），但相对顺序颠倒。

- **可采纳性**：是，因为解决一个线性冲突至少需要额外的 2 步
- **一致性**：是
- **精确度**：比曼哈顿距离更准确

2. 关键代码展示（可选）

```
def IDA_star(puzzle, heuristic_func="manhattan"):
"""
IDA*算法解决 15-Puzzle 问题
"""
solver = PuzzleSolver()
heuristic = solver.manhattan_distance

bound = heuristic(start_state)

def search(state_tuple, g, bound, path):
    nonlocal nodes_expanded

    current_state = [list(row) for row in state_tuple]
    f = g + heuristic(current_state)

    if f > bound:
        return f, None

    if solver.is_goal(current_state):
        return 0, path
```

```
        min_cost = float('inf')
        neighbors, moves = solver.get_neighbors(current_state)

        for next_state, moved_number in zip(neighbors, moves):
            nodes_expanded += 1
            new_path = path + [moved_number]
            t, result = search(next_tuple, g + 1, bound, new_path)

            if result is not None:
                return 0, result
            min_cost = min(min_cost, t)

        return min_cost, None

    while True:
        t, result = search(start_tuple, 0, bound, [])
        if result is not None:
            return result
        if t == float('inf'):
            return []
        bound = t
```

3. 创新点&优化（如果有）

三、 实验结果及分析

1. 实验结果展示示例（可图可表可文字，尽量可视化）

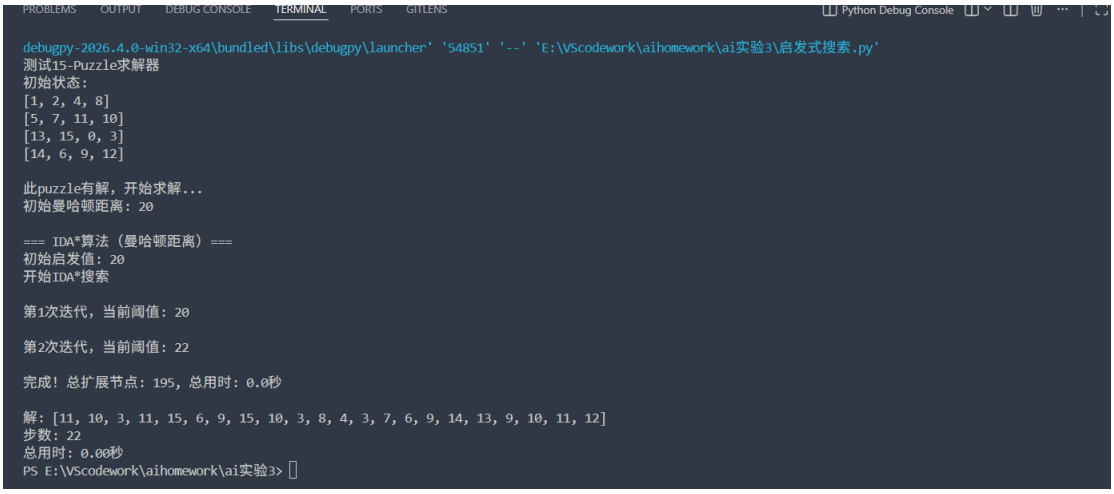
可直接复制图片粘贴在此处.

```
初始状态:
[1, 2, 3, 4]
[5, 6, 7, 8]
[9, 10, 11, 12]
[0, 13, 14, 15]

=== A*算法 (曼哈顿距离) ===
解: [13, 14, 15]
步数: 3
时间: 0.0001秒

=== IDA*算法 (曼哈顿距离) ===
解: []
步数: 0
时间: 0.0001秒

=== A*算法 (线性冲突) ===
解: [13, 14, 15]
步数: 3
时间: 0.0002秒
PS E:\VScodework\aihomework\ai实验3>
```



测试 15-Puzzle 求解器
初始状态:
[14, 10, 6, 0]
[4, 9, 1, 8]
[2, 3, 5, 11]
[12, 13, 7, 15]

此 puzzle 有解，开始求解...
初始曼哈顿距离: 35

=== IDA*算法（曼哈顿距离）===
初始启发值: 35
开始 IDA*搜索

第 1 次迭代，当前阈值: 35

第 2 次迭代，当前阈值: 37

第 3 次迭代，当前阈值: 39

第 4 次迭代，当前阈值: 41

第 5 次迭代，当前阈值: 43

第 6 次迭代，当前阈值: 45
迭代深度: 45，已扩展节点: 297049，已用时: 4.0 秒
第 7 次迭代，当前阈值: 47
迭代深度: 47，已扩展节点: 1046032，已用时: 14.0 迭代深度: 47，已扩展节点: 1197282，已用时: 16.0 迭代深度: 47，已扩展节点: 1344677，已用时: 18.0 迭代深度: 47，已扩展节点: 1498498，已用时: 20.0 迭代深度: 47，已扩展节点: 1652586，已用时: 22.0 迭代深度: 47，已扩展节点: 1806714，已用时: 24.0 迭代深度: 47，已扩展节点: 1960706，已用时: 26.0 迭代深

度: 47, 已扩展节点: 2108910, 已用时: 28.0迭代深度: 47, 已扩展节点: 2257338, 已用时: 30.0秒

第 8 次迭代, 当前阈值: 49
迭代深度: 49, 已扩展节点: 11110928, 已用时: 148.0 秒
完成! 总扩展节点: 11115792, 总用时: 148.1 秒

解: [6, 10, 9, 4, 14, 9, 4, 1, 10, 4, 1, 3, 2, 14, 9, 1, 3, 2, 13, 12, 14, 13, 5, 11, 8, 6, 4, 3, 2, 5, 12, 7, 11, 12, 7, 14, 13, 9, 5, 10, 6, 8, 12, 7, 10, 6, 7, 11, 15]
步数: 49
总用时: 148.07 秒

测试 15-Puzzle 求解器
初始状态:
[5, 1, 3, 4]
[2, 7, 8, 12]
[9, 6, 11, 15]
[0, 13, 10, 14]

此 puzzle 有解, 开始求解...
初始曼哈顿距离: 15

=== IDA*算法（曼哈顿距离）===
初始启发值: 15
开始 IDA*搜索

第 1 次迭代, 当前阈值: 15

完成! 总扩展节点: 33, 总用时: 0.0 秒

解: [13, 10, 14, 15, 12, 8, 7, 2, 5, 1, 2, 6, 10, 14, 15]
步数: 15
总用时: 0.00 秒
PS E:\VScodework\aihomework\ai 实验 3>

测试 15-Puzzle 求解器
初始状态:
[0, 5, 15, 14]

[7, 9, 6, 13]
[1, 2, 12, 10]
[8, 11, 4, 3]

此 puzzle 有解，开始求解...
初始曼哈顿距离: 42

=== IDA*算法（曼哈顿距离）===
初始启发值: 42
开始 IDA*搜索

第 1 次迭代，当前阈值: 42

第 2 次迭代，当前阈值: 44

第 3 次迭代，当前阈值: 46

第 4 次迭代，当前阈值: 48

第 5 次迭代，当前阈值: 50

第 6 次迭代，当前阈值: 52
迭代深度: 52，已扩展节点: 141932，已用时: 2.0 秒
第 7 次迭代，当前阈值: 54
迭代深度: 54，已扩展节点: 1016865，已用时: 14.0 秒
第 8 次迭代，当前阈值: 56
迭代深度: 56，已扩展节点: 7945561，已用时: 106.0 秒
第 9 次迭代，当前阈值: 58
迭代深度: 58，已扩展节点: 54362950，已用时: 716.0 秒
第 10 次迭代，当前阈值: 60
迭代深度: 60，已扩展节点: 97608842，已用时: 1282.0 秒

初始状态:
[11, 3, 1, 7]
[4, 6, 8, 2]
[15, 9, 10, 13]
[14, 12, 5, 0]
迭代深度: 54，已扩展节点: 103502410，已用时: 1372.0 秒

最后两个测试用例用时过长，半途电脑未响应。

2. 评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

|----------如有优化，请重复 1，2，分析优化后的算法结果----------|

四、 参考资料

将参考的代码,算法思路的来源表明在此处(可以是网址, 参考文献等)

PS：可以自己设计报告模板，但是内容必须包括上述的几个部分，不需要写实验感想